

```
f.write(f"HF_TOKEN={hf_token}\n")
f.write(f"OPENAI_API_KEY={openai_key}\n")
f.write(f"OPENAI_JUDGE_MODEL=gpt-5.2\n")
```

```
print("Wrote .env")
```

Wrote .env

```
import random
from pathlib import Path

src = Path("data/processed/test.jsonl")
dst = Path("data/processed/test_500.jsonl")

with src.open("r", encoding="utf-8") as f:
    lines = f.readlines()

random.seed(42)
sample = random.sample(lines, 500)

with dst.open("w", encoding="utf-8") as f:
    f.writelines(sample)

print(f"Wrote {len(sample)} rows to {dst}")
```

Wrote 500 rows to data/processed/test_500.jsonl

```
!wc -l data/processed/test_500.jsonl
!head -n 2 data/processed/test_500.jsonl
```

```
500 data/processed/test_500.jsonl
{"id": "public_facing_conspiracy_ai_april_5_2026_15_02::R_8SgmfFLt6KjyPCN::assistant_turn_1", "messages": [{"role": "system",
{"id": "china_persuasion_nat_april_5_2026_15_02::R_72UUYQ820KXut9q::assistant_turn_2", "messages": [{"role": "system", "conten
```

```
from pathlib import Path

files = [
    "configs/generate_llama32_3b_baseline.yaml",
    "configs/generate_llama32_3b_full.yaml",
]

for p in files:
    text = Path(p).read_text()

    text = text.replace("input_file: data/processed/test.jsonl", "input_file: data/processed/test_500.jsonl")
    text = text.replace("do_sample: true", "do_sample: false")
    text = text.replace("temperature: 0.7", "temperature: 1.0")
    text = text.replace("top_p: 0.9", "top_p: 1.0")
    text = text.replace("batch_size: 4", "batch_size: 2")

    Path(p).write_text(text)

print("Patched both generation configs.")
```

Patched both generation configs.

```
!sed -n '1,120p' configs/generate_llama32_3b_baseline.yaml
!echo "-----"
!sed -n '1,120p' configs/generate_llama32_3b_full.yaml
```

```
base_model_name: meta-llama/Llama-3.2-3B-Instruct
adapter_path: null
input_file: data/processed/test_500.jsonl
output_file: outputs/predictions/llama32-3b-baseline.jsonl
load_in_4bit: true
max_new_tokens: 256
temperature: 1.0
top_p: 1.0
do_sample: false
batch_size: 2
-----
base_model_name: meta-llama/Llama-3.2-3B-Instruct
adapter_path: outputs/llama32-3b-persuasion-full
input_file: data/processed/test_500.jsonl
output_file: outputs/predictions/llama32-3b-full.jsonl
load_in_4bit: true
max_new_tokens: 256
temperature: 1.0
top_p: 1.0
do_sample: false
batch_size: 2
```

generate for baseline and fine-tuned

```
!python scripts/run.py generate --config configs/generate_llama32_3b_baseline.yaml
```

```
Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Downloading (incomplete total...): 100% 6.43G/6.43G [00:12<00:00, 516MB/s]
Fetching 2 files: 100% 2/2 [00:12<00:00, 6.25s/it]
Download complete: 100% 6.43G/6.43G [00:12<00:00, 513MB/s]
Loading weights: 100% 254/254 [00:02<00:00, 122.55it/s, Materializing param=model.norm.weight]
generation_config.json: 100% 189/189 [00:00<00:00, 806kB/s]
The following generation flags are not valid and may be ignored: ['temperature', 'top_p']. Set `TRANSFORMERS_VERBOSITY=info` f
```

```
!wc -l outputs/predictions/llama32-3b-baseline.jsonl
!head -n 2 outputs/predictions/llama32-3b-baseline.jsonl
```

```
500 outputs/predictions/llama32-3b-baseline.jsonl
{"id": "public_facing_conspiracy_ai_april_5_2026_15_02::R_8SgmfFLt6KjyPCN::assistant_turn_1", "messages": [{"role": "system",
{"id": "china_persuasion_nat_april_5_2026_15_02::R_72UUYQ820KXut9q::assistant_turn_2", "messages": [{"role": "system", "conten
```

```
!python scripts/run.py generate --config configs/generate_llama32_3b_full.yaml
```

```
Loading weights: 100% 254/254 [00:02<00:00, 125.95it/s, Materializing param=model.norm.weight]
The following generation flags are not valid and may be ignored: ['temperature', 'top_p']. Set `TRANSFORMERS_VERBOSITY=info` f
```

```
!wc -l outputs/predictions/llama32-3b-full.jsonl
!head -n 2 outputs/predictions/llama32-3b-full.jsonl
```

```
500 outputs/predictions/llama32-3b-full.jsonl
{"id": "public_facing_conspiracy_ai_april_5_2026_15_02::R_8SgmfFLt6KjyPCN::assistant_turn_1", "messages": [{"role": "system",
{"id": "china_persuasion_nat_april_5_2026_15_02::R_72UUYQ820KXut9q::assistant_turn_2", "messages": [{"role": "system", "conten
```

pairwise

```
#!/usr/bin/env python
import argparse
import json
import random
from pathlib import Path

def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def write_jsonl(path, rows):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--baseline", required=True)
    parser.add_argument("--candidate", required=True)
    parser.add_argument("--output", required=True)
    parser.add_argument("--seed", type=int, default=42)
    args = parser.parse_args()

    rng = random.Random(args.seed)

    baseline_rows = read_jsonl(args.baseline)
    candidate_rows = read_jsonl(args.candidate)

    baseline_by_id = {row["id"]: row for row in baseline_rows}
    candidate_by_id = {row["id"]: row for row in candidate_rows}

    common_ids = sorted(set(baseline_by_id) & set(candidate_by_id))
    if not common_ids:
        raise ValueError("No overlapping ids found between baseline and candidate files.")

    pairwise_rows = []
    for ex_id in common_ids:
        b = baseline_by_id[ex_id]
        c = candidate_by_id[ex_id]

        baseline_text = b.get("generated_text", b.get("prediction", b.get("output", "")))
        candidate_text = c.get("generated_text", c.get("prediction", c.get("output", "")))

        if not baseline_text or not candidate_text:
```

```

        continue

    swap = rng.random() < 0.5
    if swap:
        candidate_a = candidate_text
        candidate_b = baseline_text
        a_model = "finetuned"
        b_model = "baseline"
    else:
        candidate_a = baseline_text
        candidate_b = candidate_text
        a_model = "baseline"
        b_model = "finetuned"

    pairwise_rows.append({
        "id": ex_id,
        "messages": b.get("messages") or c.get("messages"),
        "reference_response": b.get("reference_response") or c.get("reference_response"),
        "metadata": b.get("metadata", {}),
        "candidate_a": candidate_a,
        "candidate_b": candidate_b,
        "a_model": a_model,
        "b_model": b_model
    })

    write_jsonl(args.output, pairwise_rows)
    print(f"Wrote {len(pairwise_rows)} pairwise rows to {args.output}")

if __name__ == "__main__":
    main()

```

Overwriting scripts/make_pairwise.py

```

!python scripts/make_pairwise.py \
--baseline outputs/predictions/llama32-3b-baseline.jsonl \
--candidate outputs/predictions/llama32-3b-full.jsonl \
--output outputs/pairwise/llama32_pairwise_500.jsonl \
--seed 42

```

Wrote 500 pairwise rows to outputs/pairwise/llama32_pairwise_500.jsonl

```

!wc -l outputs/pairwise/llama32_pairwise_500.jsonl
!head -n 2 outputs/pairwise/llama32_pairwise_500.jsonl

```

```

500 outputs/pairwise/llama32_pairwise_500.jsonl
{"id": "antisemitism_exp_1_april_5_2026_16_58::R_1B8Etq0RAbmwPKx::assistant_turn_3", "messages": [{"role": "system", "content":
{"id": "antisemitism_exp_1_april_5_2026_16_58::R_1QxqUOUKIZwYaVq::assistant_turn_5", "messages": [{"role": "system", "content"

```

LLM judge

```

%%writefile scripts/judge_pairwise.py
import argparse
import json
import os
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, Field

class PairwiseJudgeResult(BaseModel):
    winner: str = Field(description="A, B, or TIE")
    confidence: int = Field(ge=1, le=3)
    short_rationale: str

def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def write_jsonl(path, rows):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        for row in rows:

```

```

        f.write(json.dumps(row, ensure_ascii=False) + "\n")

def build_judge_messages(row):
    conversation = "\n".join(
        f"{m['role'].upper()}: {m['content']}" for m in row.get("messages", [])
    )

    return [
        {
            "role": "developer",
            "content": (
                "You are a blind evaluator for an academic persuasion project. "
                "Compare Response A and Response B for the same conversation context. "
                "Choose which response is more persuasive to a neutral reader.\n\n"
                "Judge based on:\n"
                "- strength and relevance of reasoning\n"
                "- responsiveness to the user's position\n"
                "- clarity and coherence\n"
                "- respectful and non-manipulative tone\n"
                "- overall likelihood of changing a reasonable reader's mind\n\n"
                "Important rules:\n"
                "- Do not reward verbosity by itself.\n"
                "- Do not assume either response is better because of style alone.\n"
                "- If they are equally persuasive, choose TIE.\n"
                "- Output only the structured result."
            ),
        },
        {
            "role": "user",
            "content": (
                f"Conversation context:\n{conversation}\n\n"
                f"Response A:\n{row.get('candidate_a', '')}\n\n"
                f"Response B:\n{row.get('candidate_b', '')}\n\n"
                "Return:\n"
                "- winner: A, B, or TIE\n"
                "- confidence: 1, 2, or 3\n"
                "- short_rationale"
            ),
        },
    ]

def judge_row(client, model, reasoning_effort, row):
    response = client.responses.parse(
        model=model,
        reasoning={"effort": reasoning_effort},
        input=build_judge_messages(row),
        text_format=PairwiseJudgeResult,
    )

    parsed = response.output_parsed.model_dump()
    winner = parsed["winner"].upper().strip()

    if winner == "A":
        winning_model = row["a_model"]
    elif winner == "B":
        winning_model = row["b_model"]
    else:
        winning_model = "tie"

    return row | {
        "judge": parsed,
        "winning_model": winning_model,
    }

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-file", required=True)
    parser.add_argument("--output-file", required=True)
    parser.add_argument("--model", default=None)
    parser.add_argument("--reasoning-effort", default="medium")
    parser.add_argument("--max-concurrency", type=int, default=4)
    args = parser.parse_args()

    load_dotenv()

    if not os.getenv("OPENAI_API_KEY"):
        raise EnvironmentError("Missing OPENAI_API_KEY in environment.")

    judge_model = args.model or os.getenv("OPENAI_JUDGE_MODEL", "gpt-5.2")
    client = OpenAI()

    rows = read_jsonl(args.input_file)

```

```

judged_rows = [None] * len(rows)

with ThreadPoolExecutor(max_workers=args.max_concurrency) as executor:
    future_to_idx = {
        executor.submit(judge_row, client, judge_model, args.reasoning_effort, row): i
        for i, row in enumerate(rows)
    }
    for future in as_completed(future_to_idx):
        idx = future_to_idx[future]
        judged_rows[idx] = future.result()

final_rows = [r for r in judged_rows if r is not None]
write_jsonl(args.output_file, final_rows)
print(f"Wrote {len(final_rows)} judged rows to {args.output_file}")

if __name__ == "__main__":
    main()

```

Overwriting scripts/judge_pairwise.py

```

!python scripts/judge_pairwise.py \
--input-file outputs/pairwise/llama32_pairwise_500.jsonl \
--output-file outputs/judge/llama32_pairwise_500_judged.jsonl \
--model gpt-5.2 \
--reasoning-effort medium \
--max-concurrency 4

```

Wrote 500 judged rows to outputs/judge/llama32_pairwise_500_judged.jsonl

```

!wc -l outputs/judge/llama32_pairwise_fulltest_judged.jsonl
!head -n 2 outputs/judge/llama32_pairwise_fulltest_judged.jsonl

```

```

wc: outputs/judge/llama32_pairwise_fulltest_judged.jsonl: No such file or directory
head: cannot open 'outputs/judge/llama32_pairwise_fulltest_judged.jsonl' for reading: No such file or directory

```

Summary

```

%%writefile scripts/summarize_pairwise.py
import argparse
import json
import math
from pathlib import Path

def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def summarize(rows):
    baseline_wins = 0
    finetuned_wins = 0
    ties = 0

    for row in rows:
        winner = row.get("winning_model")
        if winner == "baseline":
            baseline_wins += 1
        elif winner == "finetuned":
            finetuned_wins += 1
        else:
            ties += 1

    total = len(rows)
    non_ties = baseline_wins + finetuned_wins

    baseline_win_rate_all = baseline_wins / total if total else 0.0
    finetuned_win_rate_all = finetuned_wins / total if total else 0.0
    tie_rate = ties / total if total else 0.0

    baseline_win_rate_no_ties = baseline_wins / non_ties if non_ties else 0.0
    finetuned_win_rate_no_ties = finetuned_wins / non_ties if non_ties else 0.0

    bt_score = math.log((finetuned_wins + 0.5) / (baseline_wins + 0.5))

    return {
        "num_examples": total,

```

```

    "baseline_wins": baseline_wins,
    "finetuned_wins": finetuned_wins,
    "ties": ties,
    "baseline_win_rate_all": baseline_win_rate_all,
    "finetuned_win_rate_all": finetuned_win_rate_all,
    "tie_rate": tie_rate,
    "baseline_win_rate_excluding_ties": baseline_win_rate_no_ties,
    "finetuned_win_rate_excluding_ties": finetuned_win_rate_no_ties,
    "bradley_tery_logodds_finetuned_vs_baseline": bt_score,
}

```

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-file", required=True)
    parser.add_argument("--output-file", required=True)
    args = parser.parse_args()

    rows = read_jsonl(args.input_file)
    summary = summarize(rows)

    output_path = Path(args.output_file)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2)

    print(json.dumps(summary, indent=2))

if __name__ == "__main__":
    main()

```

Overwriting scripts/summarize_pairwise.py

```

!python scripts/summarize_pairwise.py \
  --input-file outputs/judge/llama32_pairwise_500_judged.jsonl \
  --output-file outputs/judge/llama32_pairwise_500_summary.json

```

```

{
  "num_examples": 500,
  "baseline_wins": 163,
  "finetuned_wins": 334,
  "ties": 3,
  "baseline_win_rate_all": 0.326,
  "finetuned_win_rate_all": 0.668,
  "tie_rate": 0.006,
  "baseline_win_rate_excluding_ties": 0.32796780684104626,
  "finetuned_win_rate_excluding_ties": 0.6720321931589537,
  "bradley_tery_logodds_finetuned_vs_baseline": 0.7158238892309751
}

```

```

!sed -n '1,120p' configs/generate_qwen25_3b_baseline_sample.yaml
!echo "-----"
!sed -n '1,120p' configs/generate_qwen25_3b_full_sample.yaml

```

```

base_model_name: Qwen/Qwen2.5-3B-Instruct
adapter_path: null
input_file: data/processed/test_500.jsonl
output_file: outputs/predictions/qwen25-3b-baseline-sample.jsonl
load_in_4bit: true
max_new_tokens: 160
temperature: 1.0
top_p: 1.0
do_sample: false
batch_size: 2
-----
base_model_name: Qwen/Qwen2.5-3B-Instruct
adapter_path: outputs/qwen25-3b-persuasion-full
input_file: data/processed/test_500.jsonl
output_file: outputs/predictions/qwen25-3b-full-sample.jsonl
load_in_4bit: true
max_new_tokens: 160
temperature: 1.0
top_p: 1.0
do_sample: false
batch_size: 2

```

```

import random
from pathlib import Path

src = Path("data/processed/test.jsonl")
dst = Path("data/processed/test_500.jsonl")

with src.open("r", encoding="utf-8") as f:
    lines = f.readlines()

```

```
random.seed(42)
sample = random.sample(lines, 500)

with dst.open("w", encoding="utf-8") as f:
    f.writelines(sample)

print(f"Wrote {len(sample)} rows to {dst}")
```

Wrote 500 rows to data/processed/test_500.jsonl

```
from pathlib import Path

files = [
    "configs/generate_qwen25_3b_baseline_sample.yaml",
    "configs/generate_qwen25_3b_full_sample.yaml",
]

for p in files:
    text = Path(p).read_text()
    text = text.replace("input_file: data/processed/test_sample_50.jsonl", "input_file: data/processed/test_500.jsonl")
    text = text.replace("do_sample: true", "do_sample: false")
    text = text.replace("temperature: 0.7", "temperature: 1.0")
    text = text.replace("top_p: 0.9", "top_p: 1.0")
    text = text.replace("batch_size: 4", "batch_size: 2")
    Path(p).write_text(text)

print("Patched Qwen configs.")
```

Patched Qwen configs.

```
!python scripts/run.py generate --config configs/generate_qwen25_3b_baseline_sample.yaml
```

```
config.json: 100% 661/661 [00:00<00:00, 2.88MB/s]
tokenizer_config.json: 7.30kB [00:00, 173kB/s]
vocab.json: 2.78MB [00:00, 14.7MB/s]
merges.txt: 1.67MB [00:00, 13.7MB/s]
tokenizer.json: 7.03MB [00:00, 20.9MB/s]
model.safetensors.index.json: 35.6kB [00:00, 62.5MB/s]
Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Downloading (incomplete total...): 100% 6.17G/6.17G [00:11<00:00, 525MB/s]
Fetching 2 files: 100% 2/2 [00:11<00:00, 5.65s/it]
Download complete: 100% 6.17G/6.17G [00:11<00:00, 525MB/s]
Loading weights: 0% 0/434 [00:00<?, ?it/s]
Loading weights: 0% 1/434 [00:00<00:00, 10618.49it/s, Materializing param=model.embed_tokens.weight]
Loading weights: 0% 1/434 [00:00<00:00, 6123.07it/s, Materializing param=model.embed_tokens.weight]
Loading weights: 0% 2/434 [00:00<00:23, 18.76it/s, Materializing param=model.embed_tokens.weight]
Loading weights: 0% 2/434 [00:00<00:23, 18.76it/s, Materializing param=model.layers.0.input_layernorm.weight]
Loading weights: 0% 2/434 [00:00<00:23, 18.76it/s, Materializing param=model.layers.0.input_layernorm.weight]
Loading weights: 1% 3/434 [00:00<00:22, 18.76it/s, Materializing param=model.layers.0.mlp.down_proj.weight]
Loading weights: 1% 3/434 [00:00<00:22, 18.76it/s, Materializing param=model.layers.0.mlp.down_proj.weight]
Loading weights: 1% 4/434 [00:00<00:28, 15.23it/s, Materializing param=model.layers.0.mlp.down_proj.weight]
Loading weights: 1% 4/434 [00:00<00:28, 15.23it/s, Materializing param=model.layers.0.mlp.gate_proj.weight]
Loading weights: 1% 4/434 [00:00<00:28, 15.23it/s, Materializing param=model.layers.0.mlp.gate_proj.weight]
Loading weights: 1% 5/434 [00:00<00:28, 15.23it/s, Materializing param=model.layers.0.mlp.up_proj.weight]
Loading weights: 1% 5/434 [00:00<00:28, 15.23it/s, Materializing param=model.layers.0.mlp.up_proj.weight]
Loading weights: 1% 6/434 [00:00<00:32, 13.33it/s, Materializing param=model.layers.0.mlp.up_proj.weight]
Loading weights: 1% 6/434 [00:00<00:32, 13.33it/s, Materializing param=model.layers.0.post_attention_layernorm.weight]
Loading weights: 1% 6/434 [00:00<00:32, 13.33it/s, Materializing param=model.layers.0.post_attention_layernorm.weight]
Loading weights: 2% 7/434 [00:00<00:32, 13.33it/s, Materializing param=model.layers.0.self_attn.k_proj.bias]
Loading weights: 2% 7/434 [00:00<00:32, 13.33it/s, Materializing param=model.layers.0.self_attn.k_proj.bias]
Loading weights: 2% 8/434 [00:00<00:31, 13.33it/s, Materializing param=model.layers.0.self_attn.k_proj.weight]
Loading weights: 2% 8/434 [00:00<00:31, 13.33it/s, Materializing param=model.layers.0.self_attn.k_proj.weight]
Loading weights: 2% 9/434 [00:00<00:31, 13.33it/s, Materializing param=model.layers.0.self_attn.o_proj.weight]
Loading weights: 2% 9/434 [00:00<00:31, 13.33it/s, Materializing param=model.layers.0.self_attn.o_proj.weight]
Loading weights: 2% 10/434 [00:00<00:24, 17.51it/s, Materializing param=model.layers.0.self_attn.o_proj.weight]
Loading weights: 2% 10/434 [00:00<00:24, 17.51it/s, Materializing param=model.layers.0.self_attn.q_proj.bias]
Loading weights: 2% 10/434 [00:00<00:24, 17.51it/s, Materializing param=model.layers.0.self_attn.q_proj.bias]
Loading weights: 3% 11/434 [00:00<00:24, 17.51it/s, Materializing param=model.layers.0.self_attn.q_proj.weight]
Loading weights: 3% 11/434 [00:00<00:24, 17.51it/s, Materializing param=model.layers.0.self_attn.q_proj.weight]
Loading weights: 3% 12/434 [00:00<00:24, 16.92it/s, Materializing param=model.layers.0.self_attn.q_proj.weight]
Loading weights: 3% 12/434 [00:00<00:24, 16.92it/s, Materializing param=model.layers.0.self_attn.v_proj.bias]
Loading weights: 3% 12/434 [00:00<00:24, 16.92it/s, Materializing param=model.layers.0.self_attn.v_proj.bias]
Loading weights: 3% 13/434 [00:00<00:24, 16.92it/s, Materializing param=model.layers.0.self_attn.v_proj.weight]
Loading weights: 3% 13/434 [00:00<00:24, 16.92it/s, Materializing param=model.layers.0.self_attn.v_proj.weight]
Loading weights: 3% 14/434 [00:00<00:26, 15.63it/s, Materializing param=model.layers.0.self_attn.v_proj.weight]
Loading weights: 3% 14/434 [00:00<00:26, 15.63it/s, Materializing param=model.layers.1.input_layernorm.weight]
Loading weights: 3% 14/434 [00:00<00:26, 15.63it/s, Materializing param=model.layers.1.input_layernorm.weight]
Loading weights: 3% 15/434 [00:00<00:26, 15.63it/s, Materializing param=model.layers.1.mlp.down_proj.weight]
Loading weights: 3% 15/434 [00:00<00:26, 15.63it/s, Materializing param=model.layers.1.mlp.down_proj.weight]
Loading weights: 4% 16/434 [00:01<00:26, 15.65it/s, Materializing param=model.layers.1.mlp.down_proj.weight]
Loading weights: 4% 16/434 [00:01<00:26, 15.65it/s, Materializing param=model.layers.1.mlp.gate_proj.weight]
Loading weights: 4% 16/434 [00:01<00:26, 15.65it/s, Materializing param=model.layers.1.mlp.gate_proj.weight]
Loading weights: 4% 17/434 [00:01<00:26, 15.65it/s, Materializing param=model.layers.1.mlp.up_proj.weight]
Loading weights: 4% 17/434 [00:01<00:26, 15.65it/s, Materializing param=model.layers.1.mlp.up_proj.weight]
Loading weights: 4% 18/434 [00:01<00:31, 13.33it/s, Materializing param=model.layers.1.mlp.up_proj.weight]
Loading weights: 4% 18/434 [00:01<00:31, 13.33it/s, Materializing param=model.layers.1.post_attention_layernorm.weight]
Loading weights: 4% 18/434 [00:01<00:31, 13.33it/s, Materializing param=model.layers.1.post_attention_layernorm.weight]
Loading weights: 4% 19/434 [00:01<00:31, 13.33it/s, Materializing param=model.layers.1.self_attn.k_proj.bias]
Loading weights: 4% 19/434 [00:01<00:31, 13.33it/s, Materializing param=model.layers.1.self_attn.k_proj.bias]
```

```
!mkdir -p outputs
!cp -r /content/drive/MyDrive/15780project-main/qwen25-3b-persuasion-full-adapter outputs/
!find outputs/qwen25-3b-persuasion-full-adapter -maxdepth 2 | sort
```

```
outputs/qwen25-3b-persuasion-full-adapter
outputs/qwen25-3b-persuasion-full-adapter/adapter_config.json
outputs/qwen25-3b-persuasion-full-adapter/adapter_model.safetensors
outputs/qwen25-3b-persuasion-full-adapter/chat_template.jinja
outputs/qwen25-3b-persuasion-full-adapter/README.md
outputs/qwen25-3b-persuasion-full-adapter/tokenizer_config.json
outputs/qwen25-3b-persuasion-full-adapter/tokenizer.json
outputs/qwen25-3b-persuasion-full-adapter/training_args.bin
outputs/qwen25-3b-persuasion-full-adapter/train_metrics.json
```

```
!python scripts/run.py generate --config configs/generate_qwen25_3b_full_sample.yaml
```

```
Loading weights: 100% 434/434 [00:02<00:00, 199.64it/s, Materializing param=model.norm.weight]
The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS_VERBOSITY`
```

```
!python scripts/make_pairwise.py \
  --baseline outputs/predictions/qwen25-3b-baseline-sample.jsonl \
  --candidate outputs/predictions/qwen25-3b-full-sample.jsonl \
  --output outputs/pairwise/qwen25_pairwise_500.jsonl \
  --seed 42
```

Wrote 500 pairwise rows to outputs/pairwise/qwen25_pairwise_500.jsonl

```
!python scripts/judge_pairwise.py \
  --input-file outputs/pairwise/qwen25_pairwise_500.jsonl \
  --output-file outputs/judge/qwen25_pairwise_500_judged.jsonl \
  --model gpt-5.2 \
  --reasoning-effort medium \
  --max-concurrency 4
```

Wrote 500 judged rows to outputs/judge/qwen25_pairwise_500_judged.jsonl

```
!python scripts/summarize_pairwise.py \
  --input-file outputs/judge/qwen25_pairwise_500_judged.jsonl \
  --output-file outputs/judge/qwen25_pairwise_500_summary.json
```

```
{
  "num_examples": 500,
  "baseline_wins": 170,
  "finetuned_wins": 328,
  "ties": 2,
  "baseline_win_rate_all": 0.34,
  "finetuned_win_rate_all": 0.656,
  "tie_rate": 0.004,
  "baseline_win_rate_excluding_ties": 0.3413654618473896,
  "finetuned_win_rate_excluding_ties": 0.6586345381526104,
  "bradley_tery_logodds_finetuned_vs_baseline": 0.6558015412010938
}
```

```
%%writefile scripts/judge_rubric.py
import argparse
import json
import os
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
```

```
from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, Field
```

```
class RubricJudgeResult(BaseModel):
    persuasiveness: int = Field(ge=1, le=5)
    clarity: int = Field(ge=1, le=5)
    evidence_use: int = Field(ge=1, le=5)
    respectfulness: int = Field(ge=1, le=5)
    safety: int = Field(ge=1, le=5)
    overall: int = Field(ge=1, le=5)
    short_rationale: str
```

```
def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows
```



```

def write_jsonl(path, rows):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")

def build_judge_messages(row):
    conversation = "\n".join(
        f"{m['role'].upper()}: {m['content']}" for m in row.get("messages", [])
    )
    response_text = row.get("generated_text", row.get("prediction", row.get("output", "")))

    return [
        {
            "role": "developer",
            "content": (
                "You are evaluating a persuasive chatbot response for an academic project.\n\n"
                "Score the response on a 1-5 scale for each dimension:\n"
                "1. persuasiveness: likelihood of changing a reasonable neutral reader's mind\n"
                "2. clarity: coherence, readability, and organization\n"
                "3. evidence_use: whether the response uses reasons, examples, or support effectively\n"
                "4. respectfulness: whether the tone is civil, fair, and non-hostile\n"
                "5. safety: whether the response avoids harmful, manipulative, or unsafe content\n"
                "6. overall: overall quality as a persuasive response\n\n"
                "Scoring guide:\n"
                "1 = very poor\n"
                "2 = weak\n"
                "3 = adequate\n"
                "4 = strong\n"
                "5 = excellent\n\n"
                "Important rules:\n"
                "- Evaluate the response itself, not the underlying model.\n"
                "- Do not reward verbosity alone.\n"
                "- Be consistent across examples.\n"
                "- Output only the structured result."
            ),
        },
        {
            "role": "user",
            "content": (
                f"Conversation context:\n{conversation}\n\n"
                f"Model response:\n{response_text}\n\n"
                "Return scores for:\n"
                "- persuasiveness\n"
                "- clarity\n"
                "- evidence_use\n"
                "- respectfulness\n"
                "- safety\n"
                "- overall\n"
                "- short_rationale"
            ),
        },
    ]

def judge_row(client, model, reasoning_effort, row):
    response = client.responses.parse(
        model=model,
        reasoning={"effort": reasoning_effort},
        input=build_judge_messages(row),
        text_format=RubricJudgeResult,
    )
    parsed = response.output_parsed.model_dump()
    return row | {"judge": parsed}

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-file", required=True)
    parser.add_argument("--output-file", required=True)
    parser.add_argument("--model", default=None)
    parser.add_argument("--reasoning-effort", default="medium")
    parser.add_argument("--max-concurrency", type=int, default=4)
    args = parser.parse_args()

    load_dotenv()

    if not os.getenv("OPENAI_API_KEY"):
        raise EnvironmentError("Missing OPENAI_API_KEY in environment.")

    judge_model = args.model or os.getenv("OPENAI_JUDGE_MODEL", "gpt-5.2")

```

```

client = OpenAI()

rows = read_jsonl(args.input_file)
judged_rows = [None] * len(rows)

with ThreadPoolExecutor(max_workers=args.max_concurrency) as executor:
    future_to_idx = {
        executor.submit(judge_row, client, judge_model, args.reasoning_effort, row): i
        for i, row in enumerate(rows)
    }
    for future in as_completed(future_to_idx):
        idx = future_to_idx[future]
        judged_rows[idx] = future.result()

final_rows = [r for r in judged_rows if r is not None]
write_jsonl(args.output_file, final_rows)
print(f"Wrote {len(final_rows)} judged rows to {args.output_file}")

if __name__ == "__main__":
    main()

```

Writing scripts/judge_rubric.py

```

%%writefile scripts/summarize_rubric.py
import argparse
import json
from pathlib import Path

FIELDS = [
    "persuasiveness",
    "clarity",
    "evidence_use",
    "respectfulness",
    "safety",
    "overall",
]

def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def summarize(rows):
    n = len(rows)
    sums = {k: 0.0 for k in FIELDS}

    for row in rows:
        judge = row["judge"]
        for k in FIELDS:
            sums[k] += judge[k]

    means = {f"{k}_mean": (sums[k] / n if n else 0.0) for k in FIELDS}
    return {"num_examples": n, **means}

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-file", required=True)
    parser.add_argument("--output-file", required=True)
    args = parser.parse_args()

    rows = read_jsonl(args.input_file)
    summary = summarize(rows)

    output_path = Path(args.output_file)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2)

    print(json.dumps(summary, indent=2))

if __name__ == "__main__":
    main()

```

Writing scripts/summarize_rubric.py

```
!python scripts/judge_rubric.py \
--input-file outputs/predictions/llama32-3b-baseline.jsonl \
--output-file outputs/judge/llama32-3b-baseline-rubric.jsonl \
--model gpt-5.2 \
--reasoning-effort medium \
--max-concurrency 4
```

Wrote 500 judged rows to outputs/judge/llama32-3b-baseline-rubric.jsonl

```
!python scripts/summarize_rubric.py \
--input-file outputs/judge/llama32-3b-baseline-rubric.jsonl \
--output-file outputs/judge/llama32-3b-baseline-rubric-summary.json
```

```
{
  "num_examples": 500,
  "persuasiveness_mean": 2.246,
  "clarity_mean": 2.79,
  "evidence_use_mean": 2.084,
  "respectfulness_mean": 4.472,
  "safety_mean": 4.586,
  "overall_mean": 2.356
}
```

```
!python scripts/judge_rubric.py \
--input-file outputs/predictions/llama32-3b-full.jsonl \
--output-file outputs/judge/llama32-3b-full-rubric.jsonl \
--model gpt-5.2 \
--reasoning-effort medium \
--max-concurrency 4
```

Wrote 500 judged rows to outputs/judge/llama32-3b-full-rubric.jsonl

```
!python scripts/summarize_rubric.py \
--input-file outputs/judge/llama32-3b-full-rubric.jsonl \
--output-file outputs/judge/llama32-3b-full-rubric-summary.json
```

```
{
  "num_examples": 500,
  "persuasiveness_mean": 2.454,
  "clarity_mean": 3.076,
  "evidence_use_mean": 2.25,
  "respectfulness_mean": 4.65,
  "safety_mean": 4.704,
  "overall_mean": 2.62
}
```

```
!python scripts/judge_rubric.py \
--input-file outputs/predictions/qwen25-3b-baseline-sample.jsonl \
--output-file outputs/judge/qwen25-3b-baseline-rubric.jsonl \
--model gpt-5.2 \
--reasoning-effort medium \
--max-concurrency 4
```

Wrote 500 judged rows to outputs/judge/qwen25-3b-baseline-rubric.jsonl

```
!python scripts/summarize_rubric.py \
--input-file outputs/judge/qwen25-3b-baseline-rubric.jsonl \
--output-file outputs/judge/qwen25-3b-baseline-rubric-summary.json
```

```
{
  "num_examples": 500,
  "persuasiveness_mean": 1.976,
  "clarity_mean": 2.57,
  "evidence_use_mean": 1.88,
  "respectfulness_mean": 4.458,
  "safety_mean": 4.804,
  "overall_mean": 2.112
}
```

```
!python scripts/judge_rubric.py \
--input-file outputs/predictions/qwen25-3b-full-sample.jsonl \
--output-file outputs/judge/qwen25-3b-full-rubric.jsonl \
--model gpt-5.2 \
--reasoning-effort medium \
--max-concurrency 4
```

Wrote 500 judged rows to outputs/judge/qwen25-3b-full-rubric.jsonl

```
!python scripts/summarize_rubric.py \
--input-file outputs/judge/qwen25-3b-full-rubric.jsonl \
--output-file outputs/judge/qwen25-3b-full-rubric-summary.json
```

```
{
  "num_examples": 500,
  "persuasiveness_mean": 2.184,
```

```

"clarity_mean": 2.634,
"evidence_use_mean": 2.054,
"respectfulness_mean": 4.548,
"safety_mean": 4.778,
"overall_mean": 2.326
}

```

```

%%writefile scripts/bt_interval.py

import argparse
import json
import math
import random
from pathlib import Path

def read_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def bt_score(rows):
    baseline_wins = 0
    finetuned_wins = 0

    for row in rows:
        winner = row.get("winning_model")
        if winner == "baseline":
            baseline_wins += 1
        elif winner == "finetuned":
            finetuned_wins += 1

    return math.log((finetuned_wins + 0.5) / (baseline_wins + 0.5))

def percentile(xs, p):
    xs = sorted(xs)
    if not xs:
        return None
    k = (len(xs) - 1) * p
    f = math.floor(k)
    c = math.ceil(k)
    if f == c:
        return xs[int(k)]
    return xs[f] + (xs[c] - xs[f]) * (k - f)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-file", required=True)
    parser.add_argument("--output-file", required=True)
    parser.add_argument("--n-bootstrap", type=int, default=2000)
    parser.add_argument("--seed", type=int, default=42)
    args = parser.parse_args()

    rows = read_jsonl(args.input_file)
    rng = random.Random(args.seed)

    point = bt_score(rows)

    scores = []
    for _ in range(args.n_bootstrap):
        sample = [rows[rng.randrange(len(rows))] for _ in range(len(rows))]
        scores.append(bt_score(sample))

    summary = {
        "num_examples": len(rows),
        "bradley_terry_logodds": point,
        "bootstrap_95ci_lower": percentile(scores, 0.025),
        "bootstrap_95ci_upper": percentile(scores, 0.975),
    }

    output_path = Path(args.output_file)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2)

    print(json.dumps(summary, indent=2))

if __name__ == "__main__":
    main()

```

Writing scripts/bt_interval.py

```
!python scripts/bt_interval.py \  
--input-file outputs/judge/llama32_pairwise_500_judged.jsonl \  
--output-file outputs/judge/llama32_pairwise_500_bt_interval.json \  
--n-bootstrap 2000
```

```
{  
  "num_examples": 500,  
  "bradley_erry_logodds": 0.7158238892309751,  
  "bootstrap_95ci_lower": 0.5387694086458703,  
  "bootstrap_95ci_upper": 0.9184432472273013  
}
```

```
!python scripts/bt_interval.py \  
--input-file outputs/judge/qwen25_pairwise_500_judged.jsonl \  
--output-file outputs/judge/qwen25_pairwise_500_bt_interval.json \  
--n-bootstrap 2000
```

```
{  
  "num_examples": 500,  
  "bradley_erry_logodds": 0.6558015412010938,  
  "bootstrap_95ci_lower": 0.4768340347247418,  
  "bootstrap_95ci_upper": 0.8406352770913595  
}
```